Tackling Food Waste with Image-based Object Detection and Recipe Recommendation

Bernardo Linhares Jazbik Chalita, Brian Lu, Karl Chaker, Matteo Bonsignore

15.776 Intensive Hands-on Deep Learning, Link to Colab

December 11, 2025

1. Problem Description

Food waste is a critical global issue, with approximately one-third of all food produced for human consumption being wasted annually. A significant portion of this waste occurs at the household level, where consumers discard ingredients that have expired or are spoilt before being used. This project addresses this problem by developing an intelligent system that helps users maximize the use of their ingredients already in their refrigerator before they go bad.

The goal is to create a recipe recommendation system that automatically identifies ingredients from photographs of a user's refrigerator and suggests recipes that can be prepared with those ingredients. By making it easy to discover what meals can be prepared from available ingredients, the system encourages users to utilize items they already have rather than purchasing new ones, reducing food waste and saving money.

The system must solve two main challenges: (1) accurately detecting and classifying multiple food items in a single refrigerator image using computer vision, and (2) intelligently matching detected ingredients to appropriate recipes through an external API (see Appendix A).

The computer vision component is particularly challenging due to the cluttered nature of refrigerator images, varying lighting conditions, occlusions, and the need to distinguish between similar-looking items (e.g., different types of cheese or meat). The project utilizes a dataset of approximately 2,100 images of grocery items annotated in COCOformat (expanded to 10,000 through data augmentation), containing 30 distinct ingredient classes including fruits, vegetables, proteins, dairy products, and baking ingredients.

2. Approach

2.1 Initial Attempts and Challenges

Initial attempts to train a Vision Transformer (ViT) model from scratch for ingredient detection were hindered by significant difficulties in achieving satisfactory accuracy levels. This experience highlighted a critical distinction between the task as demonstrated in class and the requirements of this project.

- **Task Complexity and Dataset Demands:** Unlike a binary classification task, which assigns a single label to an entire image, object detection must simultaneously solve *localization*

(predicting bounding boxes) and *classification* (identifying each object). This dual-task requirement makes detection far more complex and data-hungry. Our ingredient dataset contains ~2,100 images (plus augmentations) but spans up to 30 classes with multiple objects per image, a far more challenging setting than the 96-image, two-class classification example from the lecture.

- **Architectural and Annotation Challenges:** Standard Vision Transformers produce a single image-level prediction and require heavy architectural modifications to support multi-object detection. In contrast, the model we ultimately used is a CNN-based one-stage detector specifically designed for dense object detection using a Feature Pyramid Network and Focal Loss. Even with this more suitable architecture, the task remains demanding: detection models require bounding-box annotations for every object instance, and any missing or imprecise annotations introduce noise that significantly impacts localization performance.
- **Transfer Learning Constraints:** While transfer learning allows ViTs to perform well on small datasets for classification tasks, these benefits are far more limited for object detection unless paired with specialized detection heads. RetinaNet, despite being CNN-based rather than transformer-based, still depends on rich, high-quality annotated data to learn accurate bounding boxes and class distinctions. Without an ample annotation signal, even strong detectors struggle to generalize to multi-object, multi-class settings.

Given these challenges, the project pivoted to using a pre-trained RetinaNet model, which was specifically designed and pre-trained for object detection on large-scale datasets (COCO). RetinaNet's architecture inherently handles the localization-classification dual task, and its pre-trained weights provide spatial reasoning capabilities that are directly applicable to the detection problem. This approach proved significantly more effective than attempting to adapt Vision Transformers to detection tasks.

2.2 RetinaNet Architecture

The final system is based on RetinaNet, a one-stage object detection architecture implemented via Facebook's Detectron2 framework. RetinaNet combines the efficiency of one-stage detectors with the accuracy typically associated with two-stage approaches through its focal loss mechanism, which addresses class imbalance during training. The model architecture consists of:

- **Backbone**: ResNet-50 with Feature Pyramid Network (FPN) for multi-scale feature extraction.
- **Detection Head**: Two subnetworks for classification and bounding box regression.
- **Anchor Generation**: Multi-scale anchors at different feature pyramid levels.
- **Focal Loss**: Handles class imbalance by down-weighting easy examples and focusing on hard negatives.

The model is configured with 31 classes (30 ingredients plus an unknown class) and outputs bounding box coordinates along with confidence scores for each detected ingredient. The architecture's design allows it to detect multiple ingredients of varying sizes within a single image, making it well-suited for cluttered refrigerator scenes.

2.3 Image Preprocessing

Images are read using OpenCV in BGR format (as expected by Detectron2's DefaultPredictor). The predictor automatically handles resizing to 800px on the shorter side, normalization using ImageNet statistics, and conversion to tensor format. Because the dataset was already provided with augmented

images, no additional augmentation was applied. Furthermore, since the ingredient classes were relatively well balanced, we did not employ any oversampling or resampling techniques and retained the original class distribution.

2.4 Model Inference and Post-Processing

The model's internal confidence threshold is set to 0.01 to retrieve all potential detections. Post processing applies: (1) confidence thresholding at 0.5, (2) score rounding to 3 decimal places for hardware consistency, (3) stable sorting by confidence, and (4) selection of the highest-confidence detection per unique class. This ensures consistent results between GPU and CPU environments despite floating-point precision differences.

2.5 Recipe Recommendation and Deployment

Detected ingredients are formatted and sent to the Spoonacular API, which returns top recipes ranked by ingredient match percentage. The system is deployed as a Flask web application with image upload, real-time detection, and recipe filtering options.

3 Results

The system successfully detects ingredients from refrigerator images and provides relevant recipe recommendations. The RetinaNet-based approach, leveraging transfer learning from pre-trained models, achieves strong performance on the ingredient detection task (Fig. 1). Key performance metrics include:

- **Detection Accuracy**: High precision and recall on test set images for the 30 detectable ingredient classes, with the majority of predictions lying on the diagonal of the confusion matrix (Fig. 2).
- **Multi-Object Detection**: Successfully identifies multiple ingredients within a single cluttered refrigerator image.
- **Consistency**: Identical detection results between GPU (Colab) and CPU (local) environments through careful post-processing.
- **Recipe Relevance**: Spoonacular API integration successfully returns contextually appropriate recipes based on detected ingredients.

Figure 1. Example Prediction from the Test Set

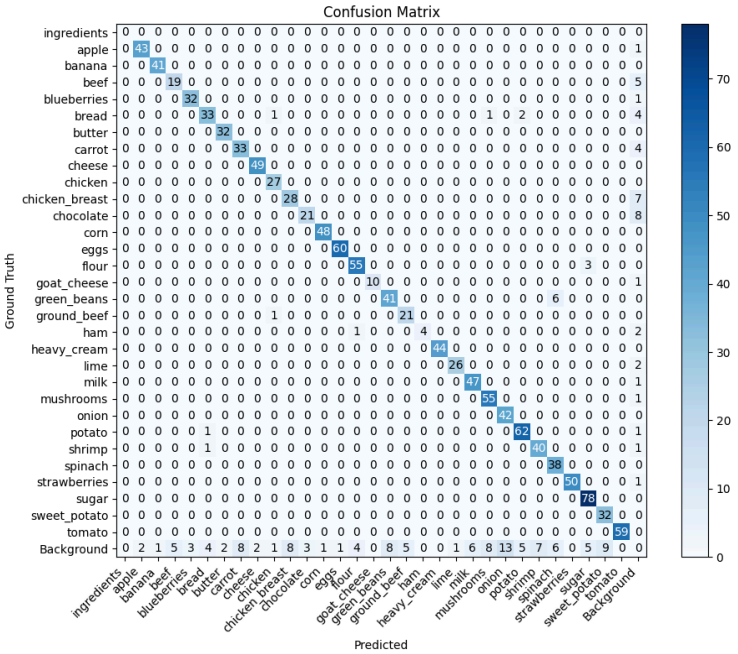

Figure 2. Confusion Matrix on the Test Set

The system successfully handles complex scenes with multiple ingredients, occlusions, and varying lighting conditions. Example detections include diverse categories: fruits (apples, strawberries), vegetables (tomatoes, carrots), proteins (ground beef, chicken), dairy (milk, cheese), and baking ingredients (flour, sugar, bread).

4 Lessons Learned

4.1 Task Complexity and Model Choice

Adapting Vision Transformers for detection highlighted that the complexity of predicting both bounding boxes and class labels far exceeds that of image classification. Transfer learning that works for small classification tasks does not directly generalize to detection. This reinforced the need to use architectures designed for detection, such as our CNN-based RetinaNet, rather than repurposing models built for simpler tasks.

4.2 Detection-Specific Challenges

Object detection introduced challenges not present in classification, including joint optimization of localization and classification, severe foreground–background imbalance, scale variation, and sensitivity to annotation quality. These factors required specialized components such as Focal Loss and multi-scale feature extraction.

4.3 Practical Implementation Factors

End-to-end development required handling API rate limits, caching, and error control, as well as ensuring consistent behavior across local and cloud environments. The interface was kept simple, allowing users to review and edit detected ingredients before recipe retrieval.

5 Summary

5.1 Next Steps

In the future, several improvements could be made to enhance the system's robustness and usability:

- **Model Robustness:** The model was trained on a relatively small dataset of approximately 2,100 unique images, and we observed a performance decline when applying it to real refrigerator images with different lighting and visual styles (Fig. 3). To improve robustness across varying conditions, such as lighting, camera angles, and fridge layouts, we plan to gather additional datasets or collect more diverse training images. Increasing the diversity of training data would help the model generalize more effectively to real-world user scenarios.
- **Recipe Recommendation Integration:** The current Spoonacular API integration is rudimentary, simply querying recipes based on detected ingredients. A more sophisticated approach would incorporate user preferences, dietary restrictions, recipe difficulty, and ingredient substitution suggestions. Better integration would also include recipe ranking algorithms that consider ingredient freshness, expiration dates, and nutritional information.
- **User Interface Enhancement:** The current Flask web application provides basic functionality but lacks polish. With more time, we would develop a more intuitive interface with features such as ingredient confidence visualization, recipe preview images, step-by-step cooking instructions, and the ability to save favorite recipes. Better error handling and user feedback mechanisms would also improve the overall user experience.

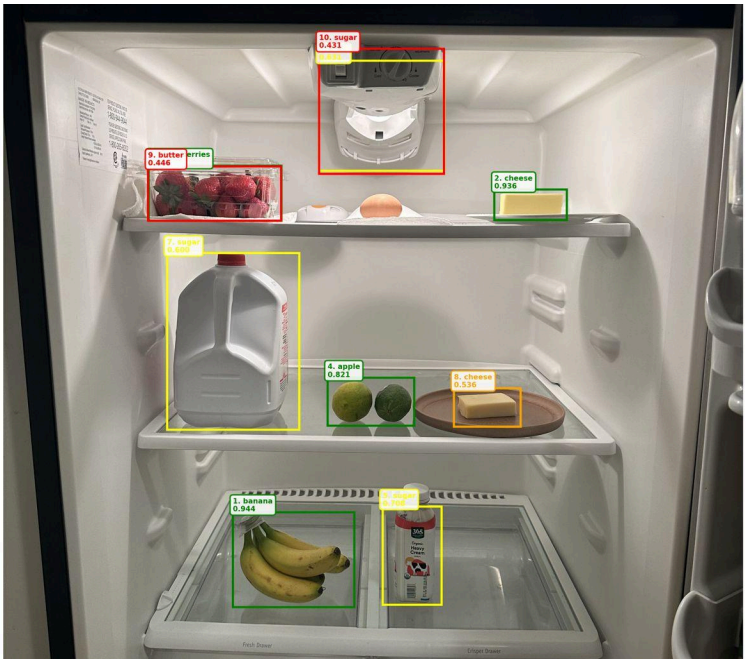

Figure 3. Detection results on our personal fridge

5.2 Conclusion

This project successfully demonstrates an end-to-end pipeline from computer vision-based ingredient detection to intelligent recipe recommendation. The transition from training Vision Transformers from scratch to leveraging pre-trained RetinaNet models illustrates the practical importance of transfer learning for real-world applications with limited datasets. The system highlights the complexity of object detection tasks and the value of proper model architecture selection.

Appendix A: Recipe Recommender User Interface Examples

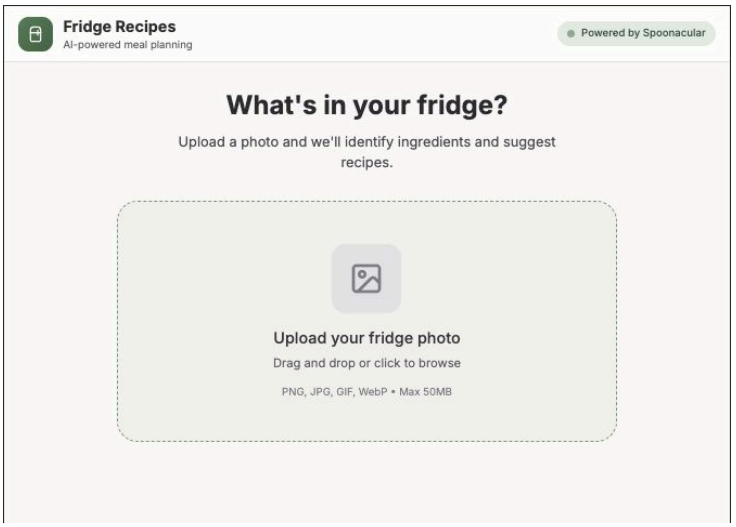

Figure 4. Recipe Recommender UI - Initial Screen

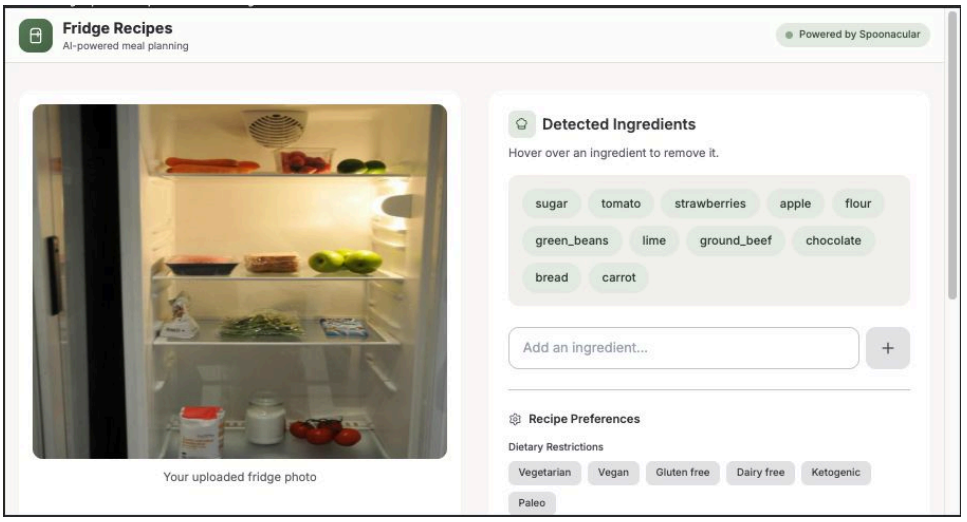

Figure 5. Recipe Recommender UI - Ingredient Detection

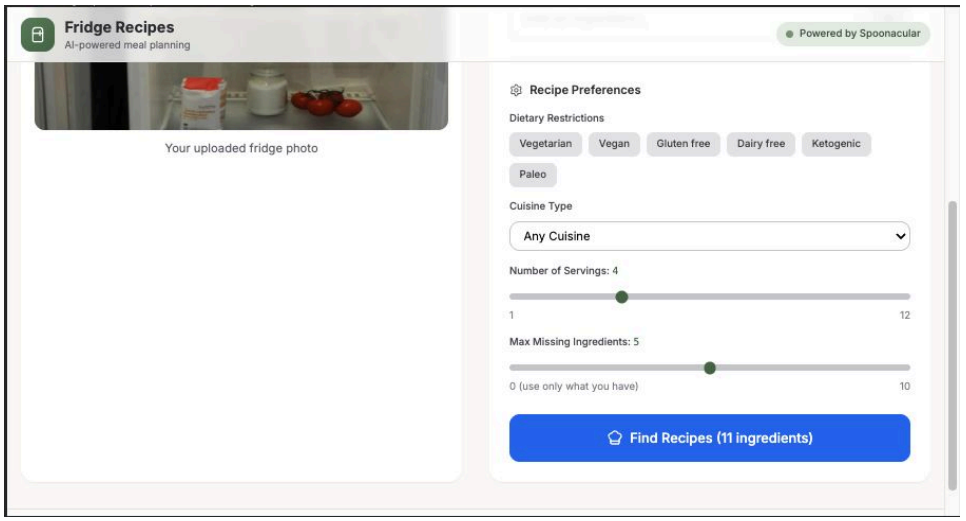

Figure 6. Recipe Recommender UI - Recipe Search Parameters